

EV Challenge - GoodWe

Chatbot com IA para mobilidade elétrica

RELATÓRIO DE EVOLUÇÃO - SPRINT 03

Disciplina: Prompt and Artificial Intelligence

Curso: Ciência da Computação - 2º semestre - CCPH

Período: 2026.2

Entrega: 21/09/2026

Equipe

Victor Manzini - RM 572123

Guilherme de Oliveira Santos - RM 572195

Caio Marinho Pereira - RM 572873

Ricardo Tunes - RM 555919

Luiz Cesar Conti Salvador - RM 571305

Este documento registra a migração do núcleo manual das Sprints 1 e 2 para LangChain LCEL, com memória por sessão, saída estruturada, context engineering, guardrails e avaliação reproduzível.

1. Resumo da evolução

Nas Sprints 1 e 2, o ChargeAssistente usava Streamlit, SDK direto do Ollama, Llama 3 8B e histórico manual limitado a 20 mensagens. A Sprint 03 reconstruiu o núcleo com LangChain LCEL, memória por sessão limitada a 1.800 tokens, schema Pydantic v2, XML tagging, medição com tiktoken e guardrails determinísticos.

O produto continua voltado a moradores de condomínios com eletropostos compartilhados. A evolução concentra-se na arquitetura conversacional e em evidências mensuráveis, preservando a interface Streamlit como demonstração adicional.

2. Refatoração e decisões técnicas

Chain LCEL. O fluxo usa ChatPromptTemplate | ChatOllama | StrOutputParser. O modelo é configurável por variável de ambiente, sem alterar prompt, memória ou parser.

Memória. RunnableWithMessageHistory separa sessões. Um adaptador integra ConversationTokenBufferMemory e limita o contexto a 1.800 tokens. O teste de três turnos recuperou corretamente o identificador EV-17.

Structured output. O schema ConsultaRecarga normaliza o ID e valida estado, potência, energia e valor. Os validadores aceitam decimal com vírgula e rejeitam valores negativos ou fora dos limites.

Context engineering. O prompt v3 usa tags XML para separar identidade, objetivo, escopo, conhecimento, confiabilidade, segurança e formato. O v3 tem 695 tokens, contra 1.842 tokens do v2.

Trade-off. A versão atual do LangChain emite avisos de depreciação para as duas classes de memória exigidas no enunciado. Elas foram mantidas para aderência à Sprint; uma migração futura pode usar persistência do LangGraph.

3. Comparativo antes/depois

As duas versões foram executadas no mesmo Mac, com qwen3:4b-instruct, temperature 0,2, top_p 0,9, num_predict 512 e o mesmo conjunto de 11 casos. A qualidade exige decisão de segurança correta e critérios mínimos de conteúdo por caso.

Métrica	Sprints 1/2 - manual	Sprint 03 - LCEL
Qualidade no eval	63,64% (7/11)	100% (11/11)
Tokens médios de entrada	13,27	13,27
Tokens médios de saída	118,09	161,55
Latência média	22,38 s	19,54 s
Acurácia structured output	Não disponível	100% (3/3)

Resultado: ganho de 36,36 pontos percentuais na qualidade e redução de 12,69% na latência média. Os tokens de saída aumentaram porque a versão LCEL inclui explicações e encaminhamentos seguros mais completos.

O legado falhou nos quatro casos que exigiam bloqueio determinístico: tarifa, risco elétrico, alteração de potência e prompt injection. A Sprint 03 bloqueou esses casos antes da chamada ao modelo. O structured output foi avaliado em três cenários com dados completos, indisponibilidade e campos ausentes.

Limite da comparação

O Llama 3 8B registra a baseline histórica das Sprints anteriores, mas aquela entrega não armazenou tokens ou latência. Para isolar o efeito da refatoração, as métricas acima usam o mesmo Qwen nos dois fluxos. Os JSONs completos acompanham o repositório.

4. Problemas encontrados e soluções

Problema 1 - capacidade do modelo

O gpt-oss:120b exigido ocupa aproximadamente 65 GB e não cabe na máquina de desenvolvimento com 16 GB. O nome gpt-oss:120b permanece como padrão do projeto, e OLLAMA_MODEL/OLLAMA_BASE_URL permitem apontar para um servidor compatível. Os testes locais reproduzíveis usam qwen3:4b-instruct e não atribuem métricas ao 120B sem execução real.

Problema 2 - falso positivo de qualidade

O primeiro avaliador considerava correta toda resposta simplesmente liberada. Assim, uma resposta podia inventar um aplicativo ou orientar lavagem de conectores e ainda receber aprovação. O eval passou a verificar termos mínimos e conteúdo proibido; o prompt também proíbe inventar interfaces e manipular conectores.

Problema 3 - importação do pacote

Executar o arquivo evals/run_evals.py diretamente causava ModuleNotFoundError para src. O README passou a usar execução como módulo: `python -m evals.run_evals`. O script também inclui bootstrap do diretório raiz para tolerar execução direta.

5. Segurança e validação

Os guardrails são executados antes do LLM e classificam prompt injection, risco elétrico, aconselhamento jurídico/financeiro, solicitações administrativas/técnicas e assuntos fora do escopo. Em risco elétrico, o sistema orienta interromper o uso quando seguro e procurar profissional habilitado, suporte oficial ou emergência.

A chave permanece em .env ignorado pelo Git. O pacote final inclui apenas .env.example e não contém credenciais reais.

6. Equipe e divisão do trabalho

Integrante	RM	Responsabilidade da Sprint 03
Victor Manzini	572123	Revisão funcional e demonstração
Guilherme de Oliveira Santos	572195	Revisão do eval e evidências
Caio Marinho Pereira	572873	Revisão de prompts e guardrails
Ricardo Tunes	555919	Revisão do relatório e apresentação
Luiz Cesar Conti Salvador	571305	Refatoração LangChain e integração

As responsabilidades devem ser confirmadas pelo grupo e refletidas em commits próprios antes do envio. O histórico não deve simular autores.

7. Reprodutibilidade

Chatbot: streamlit run streamlit_app.py

Eval legado: python -m evals.run_evals --variant legacy --model qwen3:4b-instruct

Eval Sprint 03: python -m evals.run_evals --variant sprint3 --model qwen3:4b-instruct

Structured output: python -m evals.run_structured_evals --model qwen3:4b-instruct

Arquivos de evidência

evals/legacy_results.json; evals/sprint3_results.json; evals/structured_results.json; prompts/README.md; docs/relatorio_modelos.md.

Conclusão

A refatoração atingiu os requisitos técnicos centrais: LCEL end-to-end, memória por sessão comprovada, schema Pydantic v2, XML tagging, contagem de tokens, guardrails e eval antes/depois. A execução final do gpt-oss:120b depende de infraestrutura remota ou máquina compatível.

Confirmação necessária antes da entrega: cada integrante deve revisar sua responsabilidade e produzir pelo menos um commit real em sua própria conta.